

Create virtual environment: `py -3.11 -m venv .venv`

Activates virtual environment : `.\.venv\Scripts\Activate`

Checking : `python --version`

Here's how I suggest we build it

We'll implement it **phase by phase**, just like in a real software project.

Phase 1 – Dataset Loading & EDA

We'll complete:

- `src/config.py`
- `src/data_loader.py`
- `src/preprocess.py`
- `notebooks/01_EDA.ipynb`

Features:

- Load Kaggle dataset
 - Validate schema
 - Check missing values
 - Check duplicates
 - Analyze language distribution
 - Character/word statistics
 - Save cleaned dataset
-

Phase 2 – Feature Engineering

Files:

- `src/feature_engineering.py`

We'll implement:

- Character TF-IDF (2–5)
 - Word TF-IDF (1–2)
 - Combined FeatureUnion
 - Serialization
-

Phase 3 – Model Training

Files:

- `src/train.py`
- `src/model_selection.py`
- `train_model.py`

Models:

- MultinomialNB
- LogisticRegression
- LinearSVC

We'll compare all three.

Phase 4 – Hyperparameter Tuning

GridSearchCV

Cross Validation

Model Selection

Phase 5 – Evaluation

Files:

- `src/evaluate.py`

We'll generate:

- Accuracy
 - Precision
 - Recall
 - F1
 - Confusion Matrix
 - Classification Report
-

Phase 6 – Save Pipeline

We'll save a **single Pipeline**, not separate model/vectorizer files.

```
Pipeline([
    ("features", FeatureUnion([
        ("char", char_vectorizer),
        ("word", word_vectorizer)
    ])),
    ("classifier", LinearSVC())
])
```

Phase 7 – Streamlit

We'll build



Home



Detect Language



File Detection



Batch Prediction



Model Performance



About

Professional UI.

Phase 8 – Testing

We'll test

- Empty input
 - Very short text
 - Long paragraphs
 - Mixed languages
 - Unsupported text
 - PDFs
 - DOCX
 - TXT
-

Phase 9 – Documentation

We'll create

- README
- Workflow diagrams
- Architecture diagrams
- Installation guide
- Project report (if needed)

`git remote add origin https://github.com/Hemanth-sai-G/language-detection-system.git`



Initial project structure



Added configuration module

- ✓ Implemented dataset loader
- ✓ Added preprocessing pipeline
- ✓ Completed exploratory data analysis
- ✓ Added character TF-IDF feature extraction
- ✓ Trained baseline models
- ✓ Added model comparison
- ✓ Tuned LinearSVC
- ✓ Built Streamlit homepage
- ✓ Added text prediction page
- ✓ Added file upload support
- ✓ Added batch prediction
- ✓ Improved UI and documentation

PROJECT SPECIFICATION

I am building a complete Machine Learning + Streamlit project from scratch.

This is a college project, but I want it to be implemented like a professional GitHub repository with production-quality code.

Do NOT generate shortcuts or beginner-level code.

Follow software engineering best practices.

Every file should be modular, documented, type hinted, PEP-8 compliant, reusable and production ready.

We will build the project phase by phase.

PROJECT TITLE

Language Detection System using Machine Learning

PROJECT OBJECTIVE

Detect the language of an input text with high accuracy.

The application should support:

- Typed text
- TXT file
- PDF file
- DOCX file
- Batch CSV prediction

The frontend should be developed using Streamlit.

DATASET

Dataset Name

Language Detection

Kaggle

<https://www.kaggle.com/datasets/basilb2s/language-detection>

Dataset contains

- 17 languages
- Around 10,337 samples

Languages include

English

French

Spanish

Portuguese

Italian

Russian

Dutch

Swedish

German

Danish

Greek

Turkish

Arabic

Hindi

Tamil

Kannada

Malayalam

PROJECT GOALS

The project should

- ✓ High Accuracy
- ✓ Robust
- ✓ Easy to maintain
- ✓ Professional folder structure
- ✓ Resume worthy
- ✓ GitHub worthy
- ✓ Suitable for viva
- ✓ Production quality

TECH STACK

Python 3.11

Streamlit

Scikit-learn

Pandas

NumPy

Joblib

Matplotlib

Plotly

PyPDF2

python-docx

tqdm

REQUIREMENTS.TXT

numpy==1.26.4

pandas==2.2.3

scikit-learn==1.5.2

joblib==1.4.2

streamlit==1.45.1

matplotlib==3.9.2

plotly==5.24.1

PyPDF2==3.0.1

python-docx==1.1.2

tqdm==4.67.1

DO NOT USE

TensorFlow

PyTorch

Transformers

FastText

spaCy

NLTK

Any paid APIs

WHY

The project must be built by training a model ourselves using the Kaggle dataset.

MODEL SELECTION

Train and compare

1. Multinomial Naive Bayes

2. Logistic Regression

3. Linear Support Vector Classifier (LinearSVC)

Compare baseline performance.

Choose the best model.

Expected final model

LinearSVC

FEATURE ENGINEERING

Use TF-IDF

NOT CountVectorizer.

Create

Character TF-IDF

Word TF-IDF

Combine both using FeatureUnion.

Character Vectorizer

analyzer="char"

ngram_range=(2,5)

Word Vectorizer

analyzer="word"

ngram_range=(1,2)

Use FeatureUnion to combine both.

MODEL PIPELINE

Use sklearn Pipeline.

Example

Pipeline



FeatureUnion



LinearSVC

Do NOT save separate vectorizer and model.

Save one Pipeline object.



HYPERPARAMETER TUNING

Use GridSearchCV

Tune

LinearSVC

Parameters include

classifier__C

classifier__loss

Use StratifiedKFold

5 folds

Use

scoring="f1_macro"

CROSS VALIDATION

Use

cross_val_score

5-fold Stratified CV

EVALUATION METRICS

Accuracy

Precision (macro)

Precision (weighted)

Recall (macro)

Recall (weighted)

F1 Macro

F1 Weighted

Classification Report

Confusion Matrix

Cross Validation Mean

Cross Validation Std

SAVE

Save

language_detector.pkl

Save

metrics.json

Do NOT save

vectorizer.pkl

label_encoder.pkl

Everything should be inside one sklearn Pipeline.

PREPROCESSING

Remove duplicates

Remove missing values

Trim spaces

Normalize whitespace

Do NOT

Remove punctuation

Remove accents

Remove language-specific symbols

Do NOT remove stopwords

Reason

Language detection depends heavily on punctuation, scripts and character sequences.

PROJECT STRUCTURE

Language-Detection-System/

|

| — data/

| | — raw/

| | Language Detection.csv

| | — processed/

| | cleaned_dataset.csv

| └─ sample_inputs/

|

|─ notebooks/

| |─ 01_EDA.ipynb

| |─ 02_Model_Comparison.ipynb

| └─ 03_Error_Analysis.ipynb

|

|─ src/

| |─ __init__.py

| |─ config.py

| |─ data_loader.py

| |─ preprocess.py

| |─ feature_engineering.py

| |— model_selection.py

| |— trainer.py

| |— evaluator.py

| |— predictor.py

| |— serializer.py

|

|— utils/

| |— __init__.py

| |— logger.py

| |— helpers.py

| |— pdf_reader.py

| |— docx_reader.py

| |— txt_reader.py

|

|— models/

|

|— assets/

|

|— pages/

| |— 1_Home.py

| |— 2_Text_Detection.py

| |— 3_File_Detection.py

| |— 4_Batch_Prediction.py

| |— 5_Model_Performance.py

| |— 6_About.py

|

|— app.py

|— train_model.py

└─ requirements.txt

└─ README.md

└─ .gitignore

STREAMLIT FEATURES

Home

Project overview

Workflow

Supported languages

Statistics

Text Detection

User types text

Predict language

Confidence

Prediction time

File Detection

TXT

PDF

DOCX

Batch Prediction

CSV upload

Predict every row

Download results

Performance Dashboard

Accuracy

Precision

Recall

F1

Confusion Matrix

Classification Report

Cross Validation

About

Developer

Libraries

Dataset

Workflow

CODING STANDARDS

Every file must contain

PEP-8 formatting

Type hints

Google docstrings

Comments where necessary

Exception handling

Reusable code

No duplicate code

Follow Single Responsibility Principle

GIT WORKFLOW

Every phase ends with

`git add .`

`git commit -m "Meaningful message"`

`git push`

DEVELOPMENT ORDER

Phase 1

Environment

Folder structure

Git

Dataset loading

Preprocessing

EDA

Phase 2

Feature engineering

Model comparison

Baseline models

Phase 3

Trainer

GridSearchCV

Cross Validation

Evaluation

Serializer

Predictor

Phase 4

Streamlit UI

Phase 5

File uploads

Batch prediction

Phase 6

UI improvements

Testing

README

GitHu

IMPORTANT

Do NOT generate the entire project at once.

Generate one complete file at a time.

Each file must be complete and internally consistent with the previous files.

If any earlier file needs modification due to architectural improvements, update that file before generating the next one.

Treat this as a production-grade software project rather than a collection of isolated code snippets.